



Wrocław
University
of Science
and Technology

Software Engineering

UML-based modeling of behavior

Marcin Jodłowiec

November 26th, 2025

- Introduction
- UML Behavioral Diagrams
 - Activity Diagram
 - Sequence Diagram
 - State Machine Diagram

Introduction

Introduction

Behavioral Modeling

Behavioral Modeling

Behavioral modeling captures

- ▶ how a software system acts over time,
- ▶ how components collaborate,
- ▶ how control flows, and
- ▶ how data is produced, consumed, and transformed.

The goal is to make system behavior **explicit**, **analyzable**, and **verifiable**, not implicit in prose or code.

Behavioral modeling

Stimuli

Behavioral models describe how a system responds to **stimuli** from its environment during execution. Stimuli may be of two fundamental types:

- ▶ **Data-driven stimuli:** the arrival of new data triggers processing. Systems dominated by data flow (e.g. billing or transaction processing) perform a sequence of transformations from input to output.
- ▶ **Event-driven stimuli:** discrete external or internal events cause the system to react and possibly transition to new states. Typical for real-time and control systems, where events govern behavior.

Behavioral modeling

Key aspects

- ▶ **Dynamic response:** specification of what happens after data appears or an event occurs.
- ▶ **Processing sequences:** end-to-end chains of actions from initial input to output (data-driven modeling).
- ▶ **State changes:** transitions between well-defined states caused by events and conditions (event-driven modeling).
- ▶ **Interaction structure:** ordering, causality, and coordination among components realizing a scenario.
- ▶ **Control and data flow:** how decisions, branching, and parallelism relate to the movement and transformation of data.

UML Behavioral Diagrams

UML Behavioral Diagrams

Overview

UML provides several diagram types for modeling system behavior:

- ▶ **Use Case Diagrams:** capture high-level functional requirements and interactions with actors.
- ▶ **Activity Diagrams:** model workflows, processes, and data flows.
- ▶ **Sequence Diagrams:** illustrate object interactions over time for specific scenarios.
- ▶ **State Machine Diagrams:** represent states and transitions of objects in response to events.
- ▶ **Communication Diagrams:** show interactions between objects focusing on message flow.

UML Behavioral diagrams

Comparison

UML gram	Dia- gram	Primary Type	Stimulus	What the Diagram Emphasizes
Activity Diagram		Data-driven stimuli: processing triggered by the arrival or availability of data.		End-to-end workflows, data transformations, control flow, branching, merging, and parallel processing. Represents how input data is transformed into output.
Sequence Diagram		Mixed: data stimuli (messages carrying data) and event stimuli (interaction events).		Temporal ordering of interactions, request-response patterns, synchronous vs. asynchronous calls, and the realization of a scenario through collaborating lifelines.
State Machine Diagram		Event-driven stimuli: discrete internal or external events cause state transitions.		Lifecycle modeling: states, transitions, guard conditions, entry/exit actions, and internal activities. Suitable for real-time, embedded, or safety-critical systems where behavior is state-dependent.
Communication Diagram		Mixed: interaction events (messages) with both data and event semantics.		Structural communication context: which objects collaborate, the topology of links between them, and the message exchange paths. Emphasizes <i>who</i> talks to whom rather than <i>when</i> .

Activity Diagrams: Key Ideas

- ▶ A way for modeling the **flow of work** in a system.
- ▶ Represent **sequences of actions** and how control flows between them.
- ▶ Capture both **control flow** and **data flow** at a high level.
- ▶ Suitable for describing **business processes, procedural logic, and workflow execution**.
- ▶ Emphasize the **overall process structure**, not low-level implementation.
- ▶ Effective for processes with **parallel, interleaved, or conditionally chosen** paths.

Activity diagrams

Semantics

Semantics of activity diagrams

- ▶ **token flow semantics:** tokens represent control or data moving through the activity.
- ▶ **control tokens vs object tokens:** control tokens enable the flow of control, while object tokens carry data values.
- ▶ **enabling conditions:** actions can only execute when they have the required input tokens.
- ▶ **execution semantics:** actions consume input tokens, perform processing, and produce output tokens.
- ▶ **inherent parallelism:** multiple actions can execute concurrently if their enabling conditions are met.

What Is an Activity? I

An **activity** is a parameterized behavior defined as a **coordinated flow of actions**. Execution is represented as **activity nodes** connected by **activity edges**.

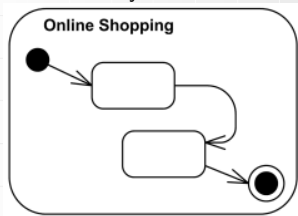
- ▶ Activity nodes represent individual execution steps:
 - ▶ **Actions** (computations, calls, object manipulation),
 - ▶ **Object nodes** (data being produced or consumed),
 - ▶ **Control nodes** (decision, merge, fork, join, etc.).
- ▶ In object-oriented models, activities are typically invoked indirectly as methods bound to operations.
- ▶ Actions may represent:
 - ▶ primitive computations (e.g. arithmetic),
 - ▶ calls to other behaviors (activities),
 - ▶ communication actions (sending signals),
 - ▶ object manipulation (reading/writing attributes or associations).

What Is an Activity? II

- ▶ Some actions explicitly **invoke activities** (call behavior action or call operation action).
- ▶ An activity is rendered as a **round-cornered rectangle** with the activity name (often in bold) and the nodes/edges inside.

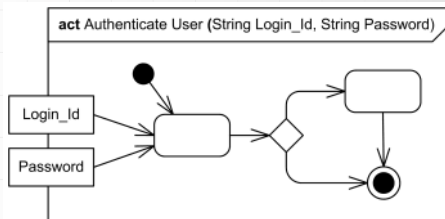
Notation:

Basic activity:

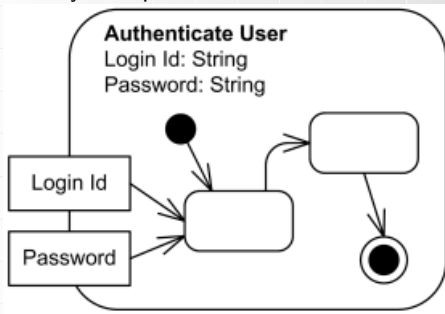


Activity with frame:

What Is an Activity? III

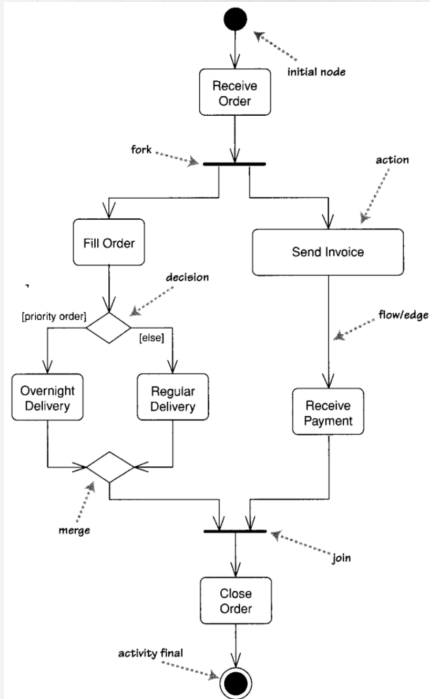


Activity with parameters:



Control Nodes and Edges

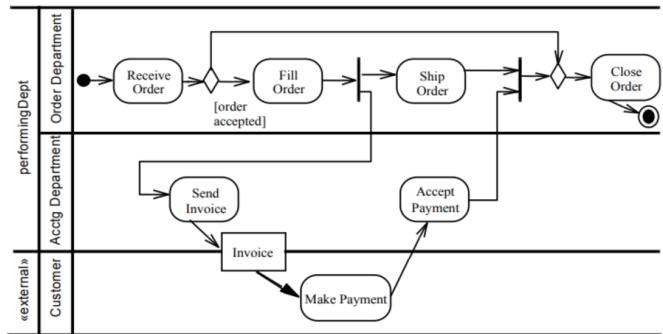
- ▶ **Decision** – one incoming flow, multiple outgoing guarded flows; selects exactly one.
- ▶ **Merge** – multiple incoming flows, one outgoing; recombines alternatives (no synchronization).
- ▶ **Fork** – one incoming flow, multiple outgoing flows; enables **parallel** execution.
- ▶ **Join** – multiple incoming flows, one outgoing; **synchronizes** parallel paths (all incoming flows must complete).
- ▶ **Activity Final** – terminates the entire activity; all executing flows stop.
- ▶ **Flow Final** – terminates a single path; other paths continue (optional if needed).
- ▶ **Edges** – connect nodes; represent **control flow** or **object flow** (data tokens moving between actions).



Activity diagrams

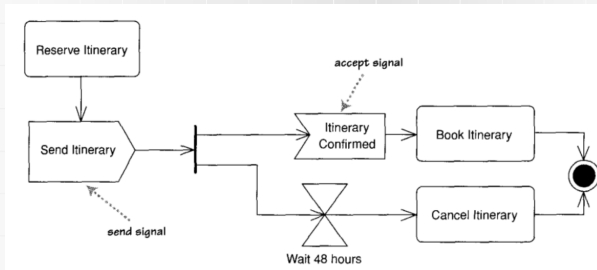
Activity partitions

- ▶ **Activity partitions** (swimlanes) group actions sharing a common characteristic (e.g. organizational role, classifier, instance, attribute value).
- ▶ They provide a constrained, role-based view of behavior; useful for modeling **responsibility**, **location**, or **ownership** of actions.



Signals

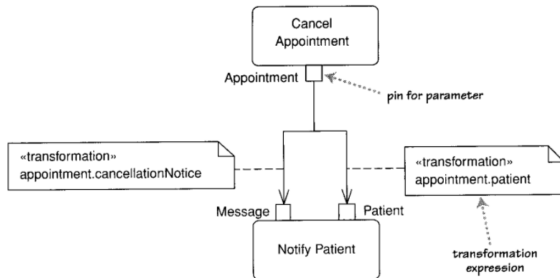
- ▶ **Signals** represent asynchronous communication between objects or components; they are sent and received without requiring a blocking response.
- ▶ A **send signal action** generates an event and delivers a signal instance to a target; a **accept event/action** waits for and consumes that signal.
- ▶ Useful for modeling loosely coupled, event-driven, or distributed behavior where control transfer is not synchronous.



Activity diagrams

Pins & transformations

- ▶ **Pins** specify the exact input and output data required by an action; they define how object tokens enter and leave the action (typed, multiplicity-controlled).
- ▶ **Object transformations** occur when output pins produce tokens that differ from the tokens consumed at input pins (e.g., computed values, converted formats).
- ▶ Pins make data flow **explicit and precise**, supporting validation of pre/postconditions and matching of object types across actions.



Sequence Diagrams: Key Ideas

- ▶ A way for modeling **interactions between objects** over time.
- ▶ Represent **message exchanges** between participants (lifelines) in temporal order.
- ▶ Capture both **synchronous** and **asynchronous** communication patterns.
- ▶ Suitable for describing **use case realizations**, **collaboration scenarios**, and **interaction protocols**.
- ▶ Emphasize the **temporal sequence** of messages, not the structural relationships.
- ▶ Effective for scenarios with **complex message ordering**, **conditional logic**, or **parallel interactions**.

Sequence diagrams

Semantics

Semantics of sequence diagrams

- ▶ **interaction semantics:** models the exchange of messages between lifelines over time.
- ▶ **lifelines vs messages:** lifelines represent participants (objects, actors, components); messages represent communication between them.
- ▶ **execution specifications:** show when an object is actively processing a message (activation bars).
- ▶ **message ordering:** messages are ordered top-to-bottom representing temporal progression.
- ▶ **synchronous vs asynchronous:** synchronous calls wait for a return; asynchronous messages do not block the sender.

Basic Elements of Sequence Diagrams I

A **sequence diagram** shows interactions between **lifelines** through **messages** arranged in temporal order.

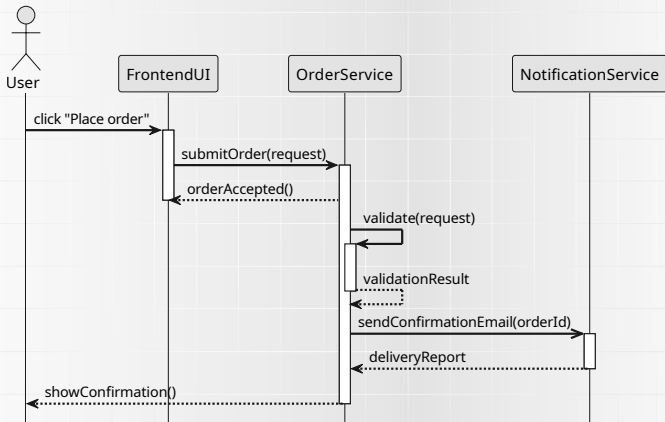
- ▶ **Lifelines** represent participants in the interaction:
 - ▶ Objects (instances of classes),
 - ▶ Actors (external entities),
 - ▶ Components or subsystems.
- ▶ Each lifeline has a **head** (rectangle with name) and a **timeline** (dashed vertical line).
- ▶ **Messages** are shown as horizontal arrows between lifelines:
 - ▶ **Synchronous message**: solid line with filled arrow (caller waits for response),
 - ▶ **Asynchronous message**: solid line with open arrow (no wait),
 - ▶ **Return message**: dashed line with open arrow (optional explicit return),

Basic Elements of Sequence Diagrams II

- ▶ **Create message:** dashed line to a lifeline head (object creation),
- ▶ **Destroy message:** ends with an X (object destruction).
- ▶ **Execution specifications** (activation bars): thin rectangles on lifelines showing when an object is executing an operation.

Basic Elements of Sequence Diagrams III

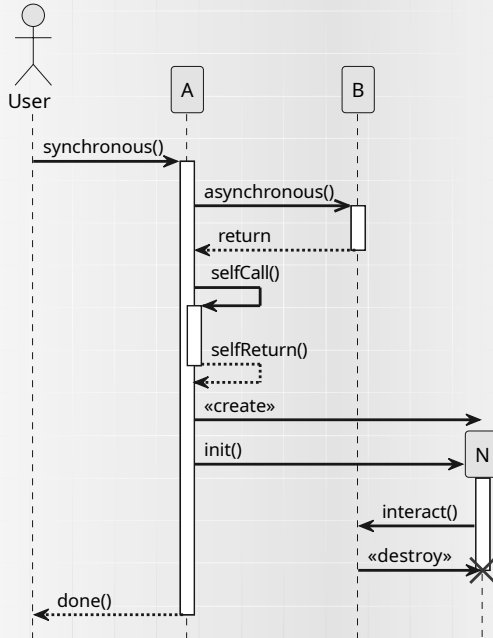
Basic Sequence Diagram - Message Types



Message Types and Interaction Patterns

- ▶ **Synchronous call** – caller blocks until the called operation completes; often paired with an explicit return message.
- ▶ **Asynchronous call** – caller continues immediately; no waiting for response.
- ▶ **Reply/Return** – explicit return from a synchronous call; may carry return values.
- ▶ **Self-call** – an object sends a message to itself (recursive or internal call).
- ▶ **Found message** – message arrives from outside the diagram scope (no identified sender).
- ▶ **Lost message** – message sent but receiver is outside diagram scope (no identified receiver).
- ▶ **Create and Destroy** – explicit lifecycle control: create instantiates a new object; destroy terminates it.

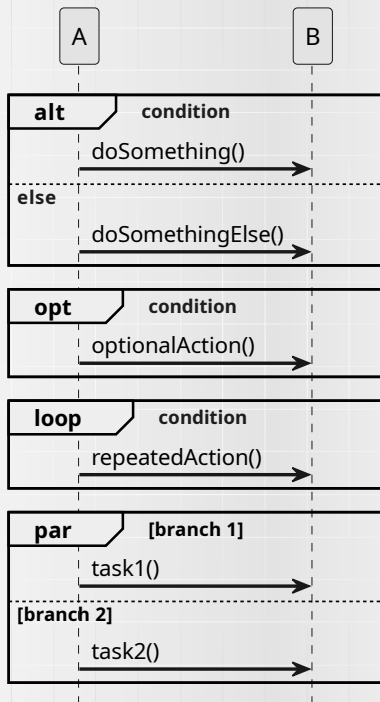
Sequence diagram: Message Types



Sequence diagrams

Combined fragments

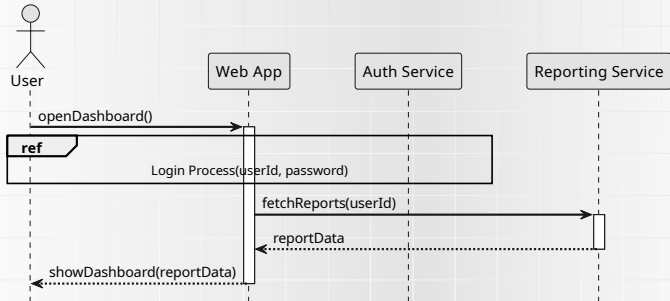
- ▶ **Combined fragments** define complex interaction logic using operators (alt, opt, loop, par, etc.).
- ▶ Each fragment has an **operator** (shown in pentagon in upper left) and one or more **operands** (regions separated by dashed lines).
- ▶ Common operators:
 - ▶ **alt**: alternatives (if-then-else logic) with guards,
 - ▶ **opt**: optional behavior (executes if guard is true),
 - ▶ **loop**: iteration (repeat while guard is true),
 - ▶ **par**: parallel execution of operands,
 - ▶ **break**: exception or early exit from enclosing fragment.
- ▶ Guards are boolean conditions written in square brackets, e.g., $[x > 0]$.



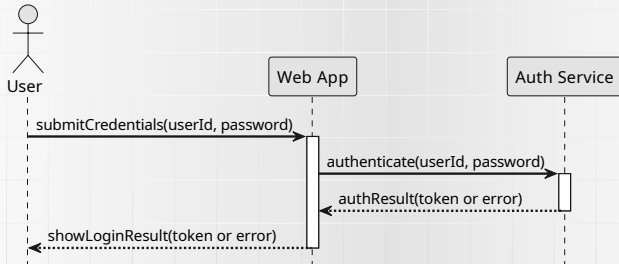
Interaction occurrences and references

- ▶ **Interaction occurrence (ref)**: references another sequence diagram to avoid repetition and improve modularity.
- ▶ Shown as a frame with **ref** in the operator pentagon and the name of the referenced interaction.
- ▶ Parameters can be passed to the referenced interaction (similar to method calls).
- ▶ Enables **decomposition** of complex scenarios into manageable, reusable pieces.
- ▶ Supports hierarchical modeling: high-level diagrams reference detailed sub-interactions.

Main Interaction with Ref to Login Process



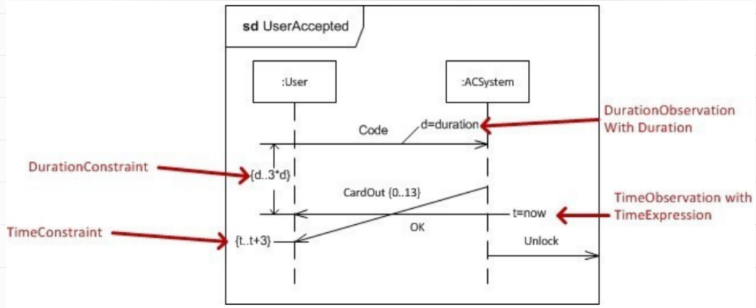
Login Process(userId, password)



Sequence diagrams

Time constraints & execution focus

- ▶ **Time constraints:** annotations specifying timing requirements (e.g., message delivery time, response deadlines).
- ▶ Shown as duration constraints (e.g., $\{t < 5s\}$) or time observations on the timeline.
- ▶ **Execution specifications** (activation bars) show the period during which an object is actively executing.
- ▶ Nested execution specifications represent call stacks or recursive invocations.
- ▶ Useful for **real-time systems, performance modeling, and systems with strict timing requirements.**



State Machine Diagrams: Key Ideas

- ▶ A way for modeling the **lifecycle and behavior** of objects in a system.
- ▶ Represent **states** that objects occupy and **transitions** between them.
- ▶ Capture **event-driven behavior** and how objects respond to stimuli.
- ▶ Suitable for describing **reactive systems, control logic,** and **protocol specifications.**
- ▶ Emphasize the **state-dependent behavior**, not the structural relationships.
- ▶ Effective for systems with **complex state-dependent logic, real-time constraints, or safety-critical requirements.**

State machine diagrams

Semantics

Semantics of state machine diagrams

- ▶ **state-based semantics:** models the lifecycle of objects as a set of states and transitions between them.
- ▶ **states vs transitions:** states represent stable conditions; transitions represent instantaneous changes.
- ▶ **event-driven execution:** transitions are triggered by events (signals, calls, time, change events).
- ▶ **guard conditions:** boolean expressions that must be true for a transition to fire.
- ▶ **run-to-completion semantics:** an event is fully processed before the next event is handled.

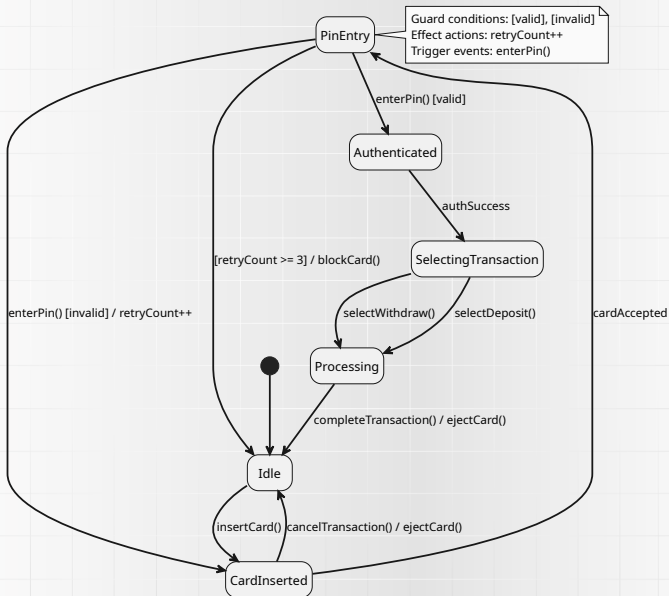
Basic Elements of State Machines I

A **state machine diagram** models the behavior of an object as a graph of **states** connected by **transitions**.

- ▶ **States** represent conditions or situations during an object's life:
 - ▶ **Simple states**: atomic states with no internal structure,
 - ▶ **Composite states**: states containing nested substates,
 - ▶ **Final states**: terminal states indicating completion.
- ▶ Each state machine has an **initial pseudostate** (filled circle) indicating where execution begins.
- ▶ **Transitions** are directed edges between states:
 - ▶ Labeled with: event [guard] / effect
 - ▶ **Event**: trigger that causes the transition,
 - ▶ **Guard**: optional boolean condition in square brackets,
 - ▶ **Effect**: optional action executed during transition.
- ▶ A state is rendered as a **rounded rectangle** with the state name.
- ▶ Transitions are shown as **solid arrows** between states.

Basic state machine diagram

Basic State Machine - ATM States



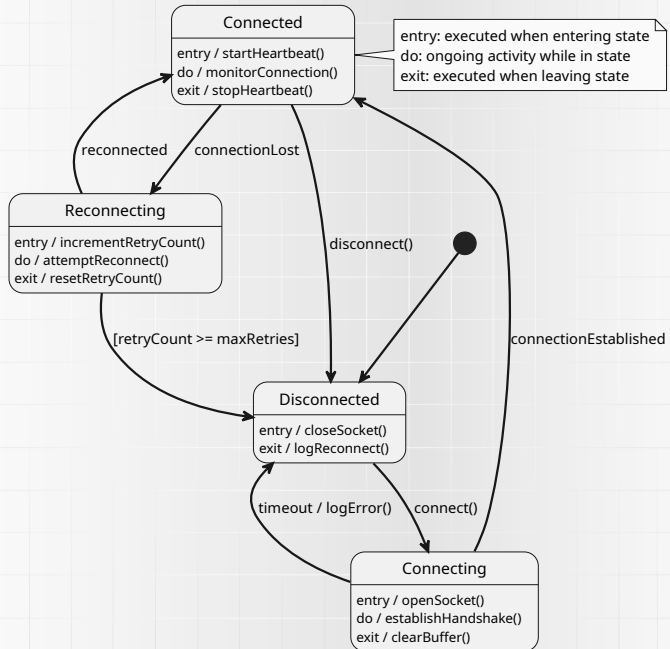
Transitions and Events

- ▶ **External transition** – leaves the source state, executes effect, enters target state (exit and entry actions executed).
- ▶ **Internal transition** – handled within the state without leaving it; no exit/entry actions executed.
- ▶ **Self-transition** – transition from a state back to itself; exit and entry actions are executed.
- ▶ **Event types:**
 - ▶ **Signal events:** asynchronous stimuli (e.g., button pressed),
 - ▶ **Call events:** synchronous operation invocations,
 - ▶ **Time events:** temporal triggers (e.g., after 5 seconds),
 - ▶ **Change events:** condition becomes true (e.g., when temperature > 100).
- ▶ **Completion event** – implicit event when all internal activities in a state complete.

State Actions and Activities

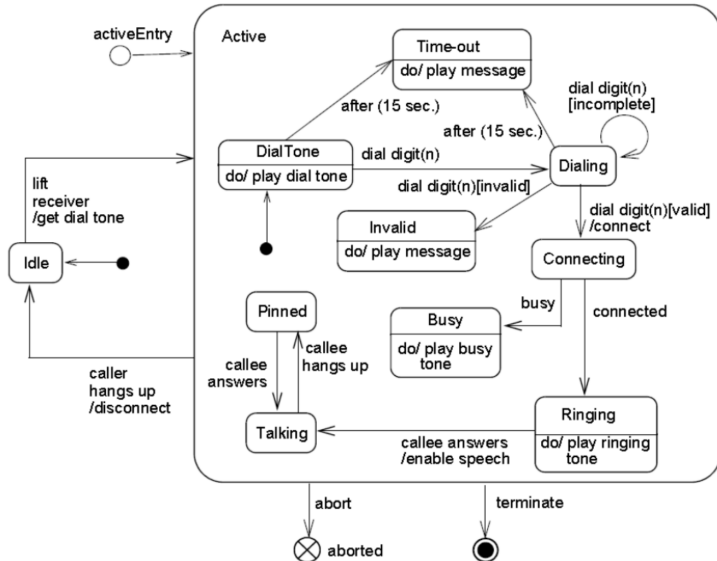
- ▶ States can define **behaviors** executed at specific points:
 - ▶ **entry** – action executed when entering the state,
 - ▶ **exit** – action executed when leaving the state,
 - ▶ **do** – ongoing activity while in the state (can be interrupted).
- ▶ **Internal transitions** – events handled without state change:
 - ▶ Notation: event [guard] / action
 - ▶ No exit/entry actions triggered.
- ▶ Actions and activities can:
 - ▶ Modify object attributes,
 - ▶ Send signals or invoke operations,
 - ▶ Access parameters and local data.
- ▶ Useful for modeling **initialization**, **cleanup**, and **continuous behavior** within states.

State Actions - Entry, Exit, and Do Activities



Composite States and Hierarchy

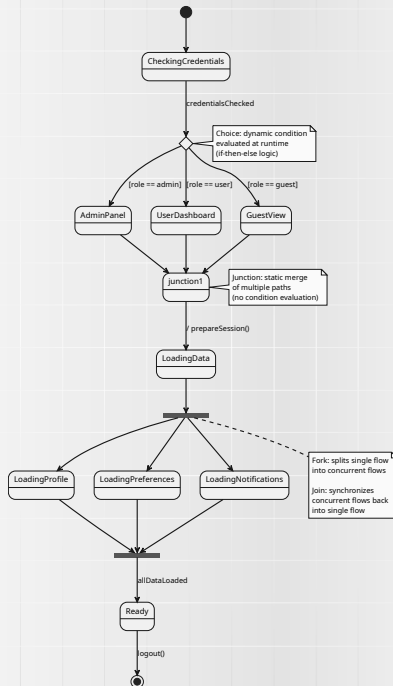
- ▶ **Composite states** contain one or more **regions** with nested substates.
- ▶ Provides **hierarchical decomposition** – complex states broken into manageable pieces.
- ▶ **Substates** inherit transitions from their container (composite state):
 - ▶ A transition from the composite state applies to all substates,
 - ▶ Enables modeling of **exception handling** and **interrupts**.
- ▶ Each region has its own **initial pseudostate** and can have a **final state**.
- ▶ When entering a composite state:
 - ▶ Default entry: transition to initial pseudostate,
 - ▶ Explicit entry: transition directly to a specific substate.
- ▶ Supports multiple levels of nesting for complex behavior modeling.



Pseudostates

- ▶ **Pseudostates** are transient nodes used for control flow – system cannot remain in a pseudostate.
- ▶ **Initial pseudostate** (filled circle): marks the starting point.
- ▶ **Choice pseudostate** (diamond): dynamic conditional branching:
 - ▶ Guards evaluated when choice is reached,
 - ▶ Exactly one outgoing transition selected (if-then-else logic).
- ▶ **Junction pseudostate** (filled circle): static merge or branch:
 - ▶ Used to combine or split transition paths,
 - ▶ Guards evaluated on incoming transitions.
- ▶ **Fork pseudostate** (thick bar): splits flow into concurrent regions.
- ▶ **Join pseudostate** (thick bar): synchronizes concurrent flows (all incoming flows must complete).

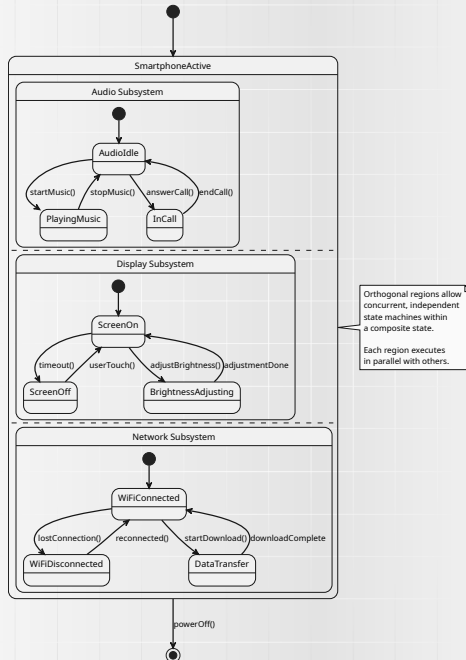
Pseudostates - Choice, Junction, Fork, and Join



Concurrent States (Orthogonal Regions)

- ▶ **Orthogonal regions** model concurrent, independent state machines within a composite state.
- ▶ Regions are separated by **dashed lines** within the composite state.
- ▶ Each region:
 - ▶ Has its own initial pseudostate and states,
 - ▶ Executes independently and concurrently with other regions,
 - ▶ Can have its own substates and transitions.
- ▶ When entering a composite state with orthogonal regions:
 - ▶ All regions become active simultaneously,
 - ▶ Each follows its own initial transition.
- ▶ Useful for modeling **parallel behavior** – e.g., multiple subsystems operating simultaneously.
- ▶ Commonly used in embedded systems, device drivers, and complex protocols.

Concurrent States - Orthogonal Regions





It's all for today!